1. What is ANN?

ANN stands for **Artificial Neural Network**. It is a type of machine-learning model that learns patterns from data.

Think about a student learning to identify a cat. You show the student many pictures of cats and non-cats. After seeing enough examples, the student learns the pattern. An ANN works in a similar way: we give it examples, and it learns patterns.

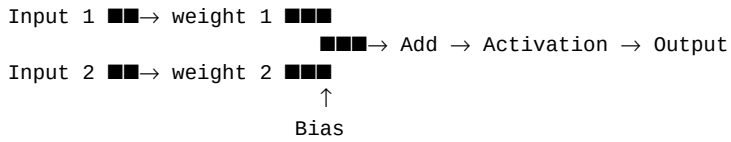
Why do we use ANN?

- To learn patterns from data.
- To make predictions.
- To solve problems where simple rules are not enough.
- It can learn complex relationships between inputs and outputs.

2. What is a Neuron?

A **neuron** is the basic unit of an ANN. You can think of it as a small calculator.

It takes inputs → gives importance to each input → adds them → gives an output.



Simple formula:

z = input × weight + bias
Then we apply an activation function to z.

Example:

```
input  = 2
weight = 0.5
bias   = 0.1

z = 2 * 0.5 + 0.1
print(z)

# Output: 1.1
```

So a neuron is basically doing calculations on the input.

3. What are Weights?

A **weight** tells the neuron how important an input is.

Imagine you want to predict whether a student will pass. You have two inputs: study hours and attendance. Maybe study hours are more important than attendance. The network can learn a bigger weight for study hours.

Easy way to remember: Weight = importance of an input.

4. What is Bias?

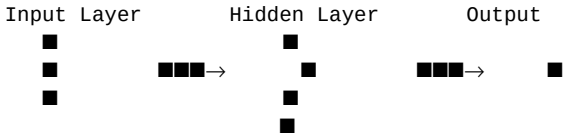
Bias is an extra value added to the calculation. It helps the neuron adjust its output more freely.

Easy way to remember: Bias = extra adjustment.

5. What is an ANN Layer?

Many neurons together form a **layer**. A simple ANN has:

- **Input layer:** receives the data.
- **Hidden layer:** learns patterns.
- **Output layer:** gives the final answer.



Example: If we have age, salary and experience, we have 3 input features. The first hidden layer can have 8 neurons. The output depends on the problem.

6. Why do we need Hidden Layers?

Hidden layers help the network learn more complicated patterns.

For example, suppose we want to decide whether a person will buy a product. The answer may depend on age + salary + previous purchases + website activity together. Hidden layers help the ANN learn these combinations.

7. What is Forward Propagation?

Forward propagation simply means **moving the data from the input side to the output side**.

```
Input
  ↓
Hidden layer
  ↓
Hidden layer
  ↓
Output
```

The network calculates something at each layer until it gets a prediction.

8. What is an Activation Function?

An activation function decides what output should come from a neuron. Most importantly, it gives the network the ability to learn **non-linear/complex patterns**.

ReLU

ReLU is very simple: if the value is negative, make it 0. If it is positive, keep it.

```
ReLU(-5) = 0
ReLU( 3) = 3
```

Sigmoid

Sigmoid gives a value between 0 and 1. This is useful for binary classification.

```
sigmoid(-5) ≈ 0.0067
sigmoid( 0) = 0.5
sigmoid( 5) ≈ 0.9933
```

Softmax

Softmax is commonly used when we have more than two classes. It gives probabilities for the classes.

For beginners, remember: **ReLU** → **hidden layers, Sigmoid** → **binary output, Softmax** → **multiple classes.**

9. What is a Prediction?

After the data passes through the network, the output neuron gives us a prediction.

Example: If we are predicting whether a customer will buy a product:

```
Output = 0.92
```

We can understand this as about 92% probability of class 1. If our threshold is 0.5, we predict class 1.

10. What is Loss?

Loss tells us **how wrong the model is**.

Example:

```
Correct answer = 1
Model prediction = 0.95
→ small error

Correct answer = 1
Model prediction = 0.10
→ large error
```

During training, the model tries to make the loss smaller.

11. How Does ANN Learn?

This is the most important part for students.

The ANN learns in a repeated cycle:

```
1. Give data to the ANN
        ↓
2. ANN makes a prediction
        ↓
3. Calculate the error (loss)
        ↓
4. Find how weights should change
        ↓
5. Update weights
        ↓
6. Repeat
```

This process happens many times. Slowly, the model becomes better at making predictions.

12. What is Backpropagation?

Backpropagation means sending the error information backward through the network to find out how the weights should be changed.

You do not need to start with difficult calculus when teaching beginners. First teach the idea:

Prediction → Error → Send error backward → Change weights → Better prediction

13. What is an Optimizer?

The optimizer decides how to change the weights using the gradients calculated during backpropagation.

Adam is a very common optimizer for beginners and practical ANN projects.

Simple idea: optimizer = tool that helps the model improve its weights.

14. What is Learning Rate?

Learning rate controls **how big the weight changes are**.

- Very small → learning can be slow.
- Very large → model may jump around and fail to learn properly.
- Good learning rate → model learns steadily.

15. What are Epoch and Batch?

Epoch: one complete pass through the training data.

Batch: a small group of training examples processed together.

Example: 1000 samples and batch size 100 means roughly 10 batches in one epoch.

16. Binary Classification vs Multiclass vs Regression

Problem	Example	Common output	Common loss
Binary	Pass / Fail	Sigmoid	Binary Cross-Entropy
Multiclass	Cat / Dog / Horse	Softmax	Categorical Cross-Entropy
Regression	House price	Linear	MSE / MAE

17. Practical ANN Code

Now build a very simple ANN for a binary classification problem.

Step 1: Import libraries

```
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

Step 2: Create simple data

```
X = np.array([
    [1, 1],
    [1, 2],
    [2, 1],
    [2, 2],
    [4, 4],
    [4, 5],
    [5, 4],
    [5, 5]
], dtype=float)

y = np.array([0, 0, 0, 0, 1, 1, 1, 1])
```

Here X has 2 features. y is our answer: 0 or 1.

Step 3: Split data

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
```

Training data is used to learn. Test data is used later to check how well the model works on unseen examples.

Step 4: Scale the data

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Scaling puts numerical features on a more comparable scale. Notice that we **fit** only on training data and then transform the test data.

18. Build the ANN

```
model = Sequential([
    Dense(8, activation="relu", input_shape=(2,)),
    Dense(4, activation="relu"),
    Dense(1, activation="sigmoid")
])
```

Read this like English:

- Start with 2 input features.
- Create a hidden layer with 8 neurons.

- Use ReLU in that layer.
- Create another hidden layer with 4 neurons.
- Finally, create 1 output neuron.
- Use sigmoid because this is binary classification.

Architecture: **2** → **8** → **4** → **1**

19. Compile the ANN

```
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

Simple meaning:

- optimizer = how the model learns
- loss = how we measure mistakes
- accuracy = how many predictions are correct

20. Train the ANN

```
history = model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=2,
    validation_split=0.2
)
```

Meaning:

- epochs=100 → go through the training data 100 times.
- batch_size=2 → use 2 examples at a time.
- validation_split=0.2 → keep part of training data for validation.

21. Test the ANN

```
test_loss, test_accuracy = model.evaluate(X_test, y_test)

print("Test loss:", test_loss)
print("Test accuracy:", test_accuracy)
```

Now we check performance on data the model did not use to learn its weights.

22. Make a New Prediction

```
new_data = np.array([[3, 3]])

new_data = scaler.transform(new_data)

probability = model.predict(new_data)

print(probability)
```

If the output is, for example, 0.91, we can interpret it as a high probability of class 1.

23. Complete Beginner Training Flow

```
Data
 ↓
Split data
 ↓
Scale data
 ↓
Build ANN
 ↓
Compile
 ↓
Train
 ↓
Check loss / accuracy
 ↓
Test
 ↓
Predict new data
```

This is the complete flow students should remember.

24. What is Overfitting?

Overfitting means the model learns the training data too closely but does not perform well on new data.

Simple example: A student memorizes all answers from one question paper but cannot solve a new paper. That is similar to overfitting.

- Training accuracy = very high
- Test/validation accuracy = much lower

Possible solutions include more data, simpler architecture, dropout, regularization, or early stopping.

25. Very Important Questions for Students

- What is ANN?
- What is a neuron?
- What is a weight?
- What is a bias?
- What is a layer?
- Why do we need activation functions?
- What is ReLU?
- Why use sigmoid for binary classification?
- What is forward propagation?
- What is loss?
- What is backpropagation?
- What is an optimizer?
- What is learning rate?
- What is an epoch?
- What is a batch?
- What is overfitting?